Dal simulatore all'hardware reale: pulsanti, LED e comunicazione seriale

Lezione 4 — 4MAK, SST-INF-01c — Programmatore, datasheet, protocollo dato+clock

28/09/2026

1 Nuovo esercizio guidato: due pulsanti e otto LED

Esempio

Traccia: scrivere un programma per PIC16F628A che rilevi lo **stato** di due **pulsanti**, A e B, secondo questa logica:

- se **nessun pulsante** è premuto, tutti gli 8 LED sono **spenti**;
- se viene premuto il pulsante A, si accendono i LED **pari**;
- se viene premuto il pulsante B, si accendono solo i 4 LED **più significativi**.

Servono quindi: un PIC16F628A, **due pulsanti in ingresso**, otto LED in uscita.

Nota

Come qualunque buon disegnatore tecnico, il primo passo prima di scrivere codice è **disegnare lo schema elettrico** alla lavagna, individuando quali piedini del microcontrollore servono e a cosa vanno collegati.

2 Dal simulatore all'hardware reale

Definizione

A differenza delle lezioni precedenti (simulatore PICSimLab), qui si lavora sul **microcontrollore fisico**: per farlo funzionare non basta il codice, serve anche sapere **come alimentarlo e come programmarlo**, informazioni che si trovano nel **datasheet** ufficiale del componente (per il PIC16F627A/628A/648A, un documento di **180 pagine** pubblicato da Microchip).

Esempio

Dal datasheet, i piedini di alimentazione e controllo del PIC16F628A:

- **VDD**: **alimentazione positiva**, **5 volt**.
- **VSS**: **massa (GND)**, **0 volt**.
- **MCLR** (*Master Clear*): piedino di reset del microcontrollore.

2.1 Il connettore di programmazione (ICSP)

Definizione

Per caricare il programma nel microcontrollore serve un **connettore di programmazione** (uno spinotto), collegato al **programmatore**: un dispositivo che si collega via USB al PC da un lato, e dall'altro al microcontrollore tramite un cavetto flat con circa 6 puntali. Il microcontrollore viene programmato **restando montato sul proprio circuito**, senza doverlo smontare.

Nota

Il nome tecnico standard di questo tipo di connessione è **ICSP** (*In-Circuit Serial Programming*): **terminologia del settore aggiunta come riferimento tecnico, non pronunciata testualmente a lezione** – **utile saperla**, ma non è stata spiegata a voce in questi termini.

Nota

Il programmatore serve **solo** per **caricare il programma**: una volta programmato, il microcontrollore può essere **staccato dal programmatore** e alimentato semplicemente con una batteria o un altro alimentatore. La scheda finale che si vende al cliente non ha bisogno del programmatore incluso (l'Arduino, per confronto, integra invece tutto – alimentazione, programmazione e microcontrollore – sulla stessa scheda).

Nota

Piedinatura tipica del connettore ICSP menzionata in aula (da verificare comunque sul datasheet specifico del programmatore usato, perché l'ordine può variare): pin 1 = MCLR, poi VDD, VSS, poi PGD e PGC sui pin 4 e 5, con un eventuale pin 6 non collegato a nulla.

Definizione

Il piedino **MCLR** va collegato a **VDD (+5V)** tramite una **resistenza di pull-up** da 10 kΩ: il valore preciso non richiede calcolo, è un **valore standard** da usare senza doverlo verificare per questo tipo di collegamento.

Attenzione

Il piedino **RA5** coincide fisicamente con **MCLR**: significa che, quando si usa la programmazione ICSP, quel piedino del gruppo **PORTA** **non è più disponibile** come input/output generico, va sempre **riservato al reset**.

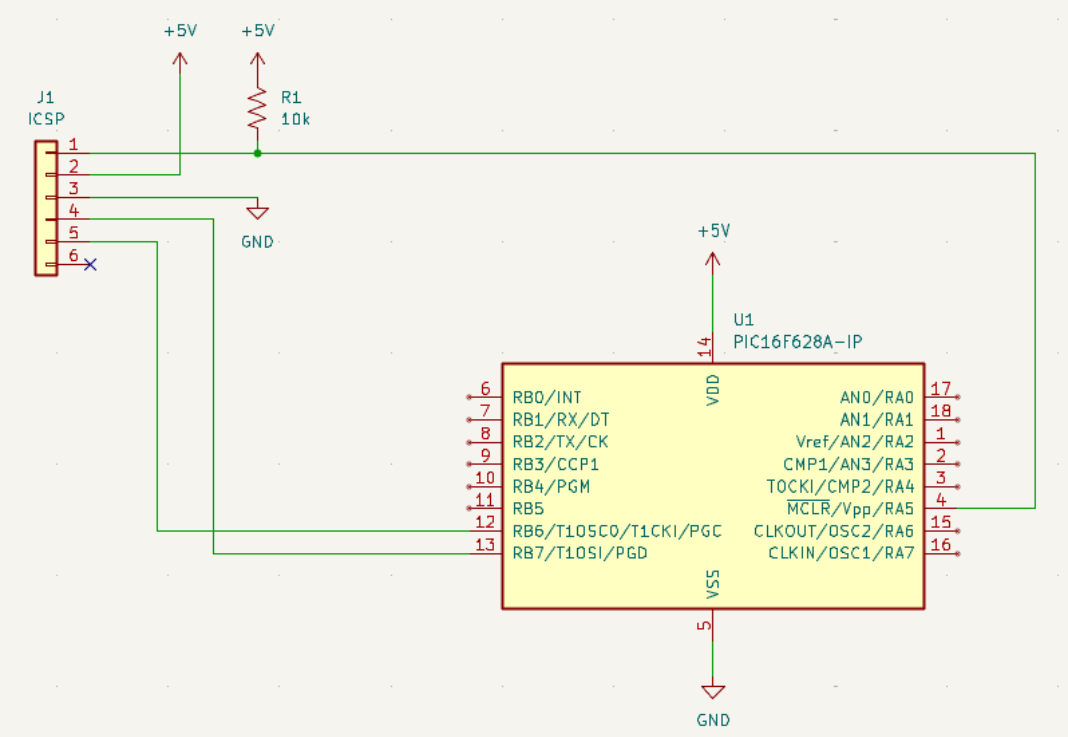


Figura 1: Piedini di alimentazione e programmazione del PIC16F628A: VDD/VSS per l'alimentazione, RA5/MCLR con resistenza di pull-up da 10 kΩ verso VDD, e RB6/PGC + RB7/PGD per la programmazione seriale ICSP tramite connettore J1 (schema KiCad).

3 I piedini di programmazione: PGD e PGC

Definizione

Gli ultimi due piedini del gruppo B, **RB7** e **RB6**, coincidono rispettivamente con **PGD** (*Program Data*) e **PGC** (*Program Clock*): sono i piedini usati dal programmatore per **trasferire il codice** nel microcontrollore, tramite una **comunicazione seriale sincrona**.

Nota

Questo è lo stesso principio di funzionamento già visto con il **registro a scorrimento 74HC595**: un filo **Data** porta un bit alla volta, un filo **Clock** scandisce quando quel bit va **"fotografato"** e memorizzato.

3.1 Come funziona: un bit alla volta, scandito dal clock

Definizione

Per trasmettere un dato in modo seriale sincrono servono due segnali:

- **Data**: porta, un bit alla volta, il valore (0 o 1) che si vuole trasmettere.
- **Clock**: un segnale che cambia stato (**fronte di salita**) ogni volta che il valore su Data va "**catturato**".

In corrispondenza di ogni **fronte di salita** del clock, il dispositivo ricevente esegue, per così dire, una fotografia del valore presente in quel momento sul filo Data, e lo memorizza. Ripetendo l'operazione un bit alla volta, si trasferisce un **intero numero binario** in sequenza.

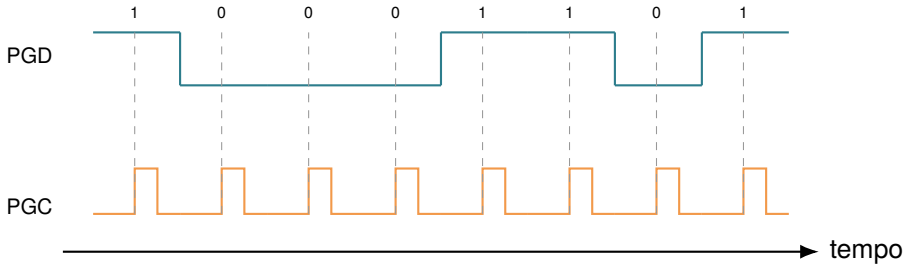


Figura 2: Comunicazione seriale sincrona: il filo Data (PGD) porta la sequenza binaria 1-0-0-0-1-1-0-1 un bit alla volta; il filo Clock (PGC) genera un impulso per ogni bit, e sul fronte di salita del clock il valore presente su Data viene "catturato" dal ricevitore.

Nota

Lo stesso principio (**dato + clock**, un bit alla volta) è alla base sia della programmazione ICSP dei PIC, sia del funzionamento del 74HC595 già studiato: è un **modo generale ed economico** di trasmettere dati in serie usando pochi fili, al costo di essere **più lento** di una trasmissione parallela (dove tutti i bit viaggiano contemporaneamente su fili distinti).

Esempio

Concretamente, quando MPLAB X compila il programma genera un **file `.hex`**: aprendolo con un editor di testo si vede una sequenza di **coppie di cifre esadecimali**. Ad esempio la coppia `FC` vale 11111100 in binario: il programmatore genera 8 colpi di clock su PGC, e per ognuno mette sul filo PGD il bit corrispondente (prima un 1, poi un altro 1, e così via, poi due 0). Il file può essere lungo, ma essendo il processo molto veloce il caricamento richiede solo pochi secondi.

4 Il quarzo: i piedini RA6 e RA7

Definizione

Oltre a MCLR, VDD, VSS, PGD e PGC, restano da collegare altri due piedini indispensabili: **RA6** e **RA7**, dedicati al **quarzo** (l'**oscillatore esterno** che genera il segnale di clock del microcontrollore). Sono collegati a un circuito con due **condensatori** da circa **18 pF** ciascuno: il valore non richiede calcolo, è uno standard (vanno bene anche condensatori da 22 pF, grazie alla tolleranza del componente).

Nota

La **frequenza del quarzo** da scegliere dipende da quanto veloce deve andare il microcontrollore: il PIC16F628A arriva al massimo a **20 MHz**. Un quarzo più veloce del limite dichiarato (es. 32 MHz) rischia di far scaldare e bruciare il componente. Quarzi da 20, 16, 10, 8, 4 o

2 MHz vanno tutti bene, a seconda della velocità desiderata.

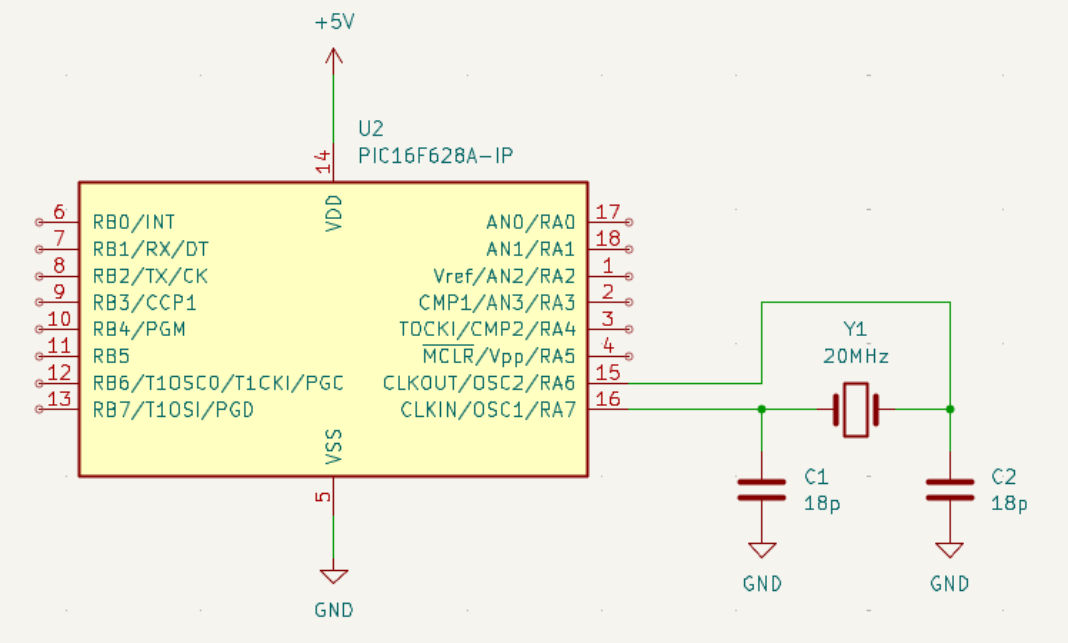


Figura 3: Collegamento del quarzo Y1 (20 MHz) sui piedini RA6/OSC2 e RA7/OSC1, con i due condensatori di carico C1/C2 da 18 pF verso massa (schema KiCad).

Esempio

Fisicamente, un quarzo si presenta come un piccolo corpo metallico (spesso a forma di piccolo cappello), con dimensioni che vanno da circa 1,5 mm per i modelli più compatti fino a cilindretti di alcuni millimetri; esistono anche versioni **SMD**, con i piedini sotto il corpo, più delicate da saldare.

5 Lo schema completo: pulsanti e LED

Nota

Riassumendo i vincoli: **RA5 è occupato da MCLR**, **RA6/RA7 sono occupati dal quarzo**, **RB6/RB7 sono occupati da PGD/PGC**. Restano liberi per l'uso generico i piedini da **RA0 a RA4** e da **RB0 a RB5** — con RB6/RB7 comunque utilizzabili aggiungendo un **resistore di protezione**, come richiesto in questo caso dal testo dell'esercizio che vuole usare tutti gli 8 piedini di **PORTB** per i LED.

Esempio

Scelta dei piedini per questo esercizio: i due **pulsanti** vengono collegati su **RA2** e **RA3** (non su RA0, perché nel circuito di laboratorio già disponibile quel piedino è cablato come uscita), e gli 8 LED (con le rispettive resistenze di limitazione di corrente) vengono collegati su tutta **PORTB** (RB0-RB7), con un **resistore aggiuntivo di protezione** sui due piedini RB6/RB7 condivisi con la programmazione.

6 Scrivere il programma: configurazione e lettura dei pulsanti

Definizione

Creato il nuovo progetto in MPLAB X (**Application Project**, PIC16F628A, compilatore XC8), la prima cosa da impostare sono i **Configuration Bits** (menu *Window* → *Target Memory View* → *Configuration Bits*). Per questo circuito con quarzo esterno vanno modificate **4 impostazioni** rispetto ai valori proposti di default:

- **FOSC = HS** (oscillatore a quarzo, High Speed);
- **WDTE = OFF** (**watchdog disattivato**: non lo gestiamo ancora);
- **PWRTE = ON** (**power-up timer attivato**);
- **LVP = OFF** (**programmazione a basso voltaggio disattivata**).

MPLAB X permette di generare automaticamente le righe di codice corrispondenti (`#pragma config ...`) da **incollare in cima al file** `main.c`.

Esempio

Impostazione degli input/output: i pulsanti su RA2 e RA3 devono essere **input**, tutto il resto di PORTA **non interessa** in questo esercizio. Visualizzando `TRISA` come una **griglia di 8 bit** (posizioni 0-7), si mettono a **1 (input)** le posizioni 2 e 3, a 0 tutte le altre:

```
TRISA = 0x0C;  // imposto RA2 e RA3 come input (0000 1100)
TRISB = 0x00;  // tutta PORTB in uscita (per i LED)
```

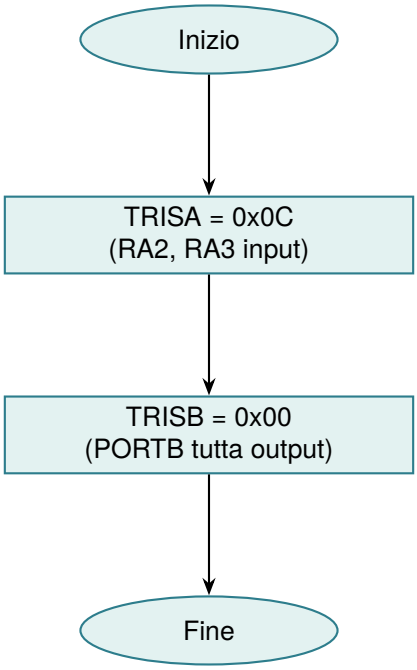


Figura 4: Diagramma di flusso della configurazione iniziale dei pin: due assegnazioni in sequenza, TRISA per gli input (pulsanti) e TRISB per gli output (LED).

Definizione

Il codice deve **girare all'infinito**, quindi tutta la logica ripetuta va dentro un **ciclo `while(1)`**: qualunque istruzione messa **fuori** dal `while` verrebbe eseguita **una sola volta all'accensione** e mai più.

Esempio

Il primo passo dentro il ciclo è **leggere lo stato dei pulsanti**, leggendo l'**intera PORTA** in una **variabile**:

```
char buttons = PORTA;
```

Mettere **`PORTA` a destra dell'uguale** effettua una **lettura** della porta; se fosse stata scritta **a sinistra dell'uguale**, sarebbe stata invece una **scrittura**.

7 Isolare i bit che interessano: l'operatore bitwise AND come maschera

Attenzione

Leggendo tutta `PORTA`, il valore contiene anche lo stato di tutti gli **altri piedini** (RA0, RA1, RA4, RA5, RA6, RA7), che a noi non interessano e potrebbero contenere valori residui imprevedibili – il fatto che un piedino sia configurato come *output* non garantisce che il suo valore letto sia zero: qualche altra parte del programma potrebbe averci scritto un 1 in precedenza.

Definizione

Per **isolare solo i bit** di RA2 e RA3, ignorando tutti gli altri, si usa l'**operatore bitwise AND** (`&`) con una **maschera**: un numero che ha 1 solo nelle posizioni che interessano e **0 in tutte le altre**. L'AND bit a bit tra un valore qualsiasi e uno 0 dà sempre 0, quindi la **maschera** "spegne" tutti i bit che non interessano, lasciando intatti solo quelli desiderati:

```
char buttons = PORTA & 0x0C;  // 0x0C = 0000 1100: isola solo RA2 e RA3
```

Conviene **applicare la maschera** direttamente nella riga di lettura, così si è sicuri di partire già da un dato filtrato.

Esempio

Dopo la maschera, i soli valori possibili di `buttons` sono quattro:

- `0x0C` (0000 1100): **entrambi i pulsanti premuti** – caso che in questo esercizio **non si presenta mai**, perché la traccia non lo prevede;
- `0x08` (0000 1000): premuto solo il pulsante collegato a **RA3**;
- `0x04` (0000 0100): premuto solo il pulsante collegato a **RA2**;
- `0x00`: **nessun pulsante premuto**.

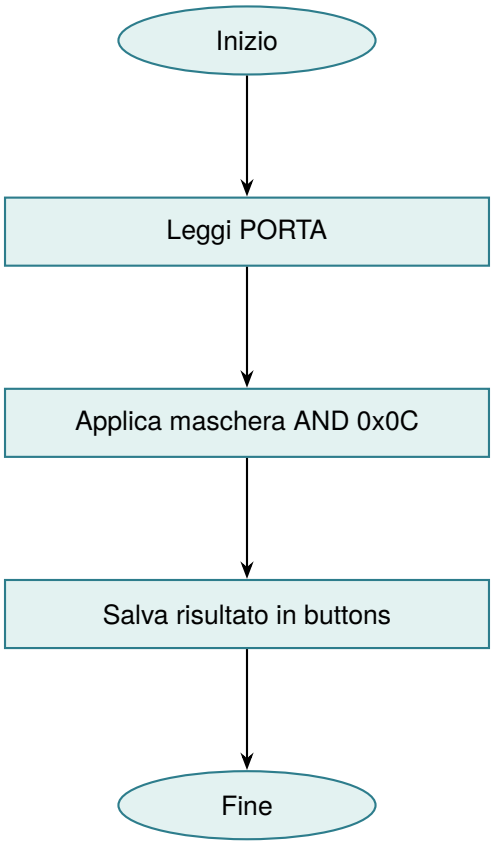


Figura 5: Diagramma di flusso della lettura filtrata: il valore di PORTA viene mascherato con l'AND bitwise prima di essere salvato, isolando solo i bit di RA2 e RA3.

8 La logica completa: if/else e i LED pari, dispari, più significativi

Esempio

Soluzione completa dell'esercizio:

```
#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config MCLRE = ON
#pragma config BOREN = ON
#pragma config LVP = OFF
#pragma config CPD = OFF
#pragma config CP = OFF

#include <xc.h>

void main(void) {
    TRISA = 0x0C;   // RA2, RA3 input (pulsanti)
    TRISB = 0x00;   // PORTB tutta output (led)

    while (1) {
        char buttons = PORTA & 0x0C;

        if (buttons == 0x00) {
            PORTB = 0x00;   // nessun pulsante: tutto spento
        }
```

```
        else if (buttons == 0x04) {
            PORTB = 0b01010101;   // pulsante A: led dispari accesi (0x55)
        }
        else if (buttons == 0x08) {
            PORTB = 0xF0;   // pulsante B: i 4 led piu' significativi accesi
        }
    }
}
```

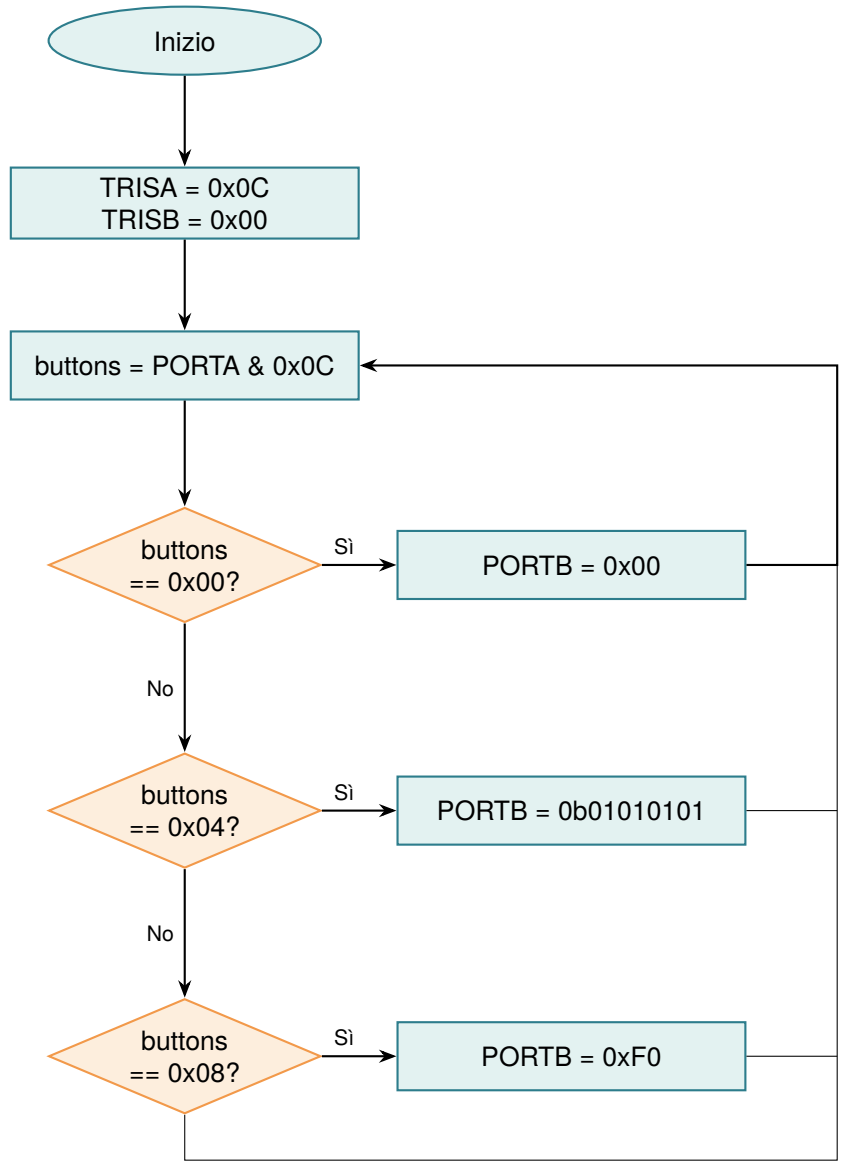


Figura 6: Diagramma di flusso del programma completo: dopo la configurazione iniziale, il ciclo `while(1)` rilegge PORTA e verifica in cascata (`if/else if`) quale pulsante è premuto, aggiornando PORTB di conseguenza, poi ricomincia da capo.

Nota

Il valore **`0b01010101`** (equivalente a `0x55`) accende i LED nelle posizioni 0, 2, 4, 6 di PORTB: contando le posizioni da 0, queste sono le posizioni **pari** – coerente con la richiesta della traccia ("LED pari" con il pulsante A). Il valore **`0xF0`** (`11110000` in binario) accende invece i **4**

bit più a sinistra di PORTB (RB7-RB4), cioè i 4 bit **più significativi**: la **cifra più a sinistra** di un numero è sempre quella che "pesa di più" (come nel numero 1243, dove l'1 vale mille ed è la cifra più significativa) – lo stesso principio si applica alle posizioni di un byte.

9 Cosa ricordare di questa lezione

- Passare dal simulatore all'hardware reale richiede di consultare il **datasheet** del componente per sapere come alimentarlo e collegarlo correttamente.
- **VDD** (+5V) e **VSS** (0V, massa) sono i piedini di alimentazione; **MCLR** è il piedino di reset, da collegare a VDD tramite una **resistenza di pull-up** da 10 kΩ.
- La programmazione **ICSP** usa **due piedini dedicati**: **PGD** (Data) e **PGC** (Clock), che sui piedini generici corrispondono a **RB7 e RB6**.
- **RA6**/**RA7** sono dedicati al **quarzo** (con due condensatori da 18 pF), la cui frequenza massima per il PIC16F628A è **20 MHz**.
- RA5/MCLR, RA6-RA7/quarzo e RB7-RB6/PGD-PGC, quando usati per queste funzioni, **non sono più disponibili** come input/output generici: **restano liberi** RA0-RA4 e RB0-RB5 (RB6/RB7 riusabili con un resistore di protezione).
- La comunicazione **seriale sincrona** (dato + clock) trasmette **un bit alla volta**: ad ogni **fronte di salita** del clock, il valore sul filo dati viene catturato – lo stesso principio già visto nel 74HC595.
- I **Configuration Bits** (oscillatore, watchdog, power-up timer, LVP) vanno impostati correttamente prima di scrivere il codice, in base al circuito reale (per un quarzo esterno: **FOSC = HS**).
- L'operatore bitwise **AND** con una **maschera** **isola** solo i bit che interessano, **azzerando tutti gli altri**: fondamentale quando si legge un'**intera porta** ma serve solo lo stato di alcuni piedini specifici.